MAPEO DE OBJETOS A BASES DE DATOS

Persistencia con JPA, Hibernate y el patrón DAO

DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS · TECNICATURA EN DESARROLLO DE SOFTWARE

Prof. Esteban Santini · Septiembre 2026

¿QUÉ ES JPA?

Java Persistence API: mapeo objeto-relacional

JPA (Java Persistence API) es la especificación estándar de Java para mapear clases a tablas y trabajar con bases de datos relacionales sin escribir SQL a mano.

En vez de armar el SQL y construir los objetos fila por fila como con JDBC, en JPA se anotan las clases (@Entity, @Id, @Column) y el proveedor genera las consultas automáticamente a partir de esas anotaciones.

JPA es solo una especificación (un conjunto de interfaces, como jakarta.persistence); necesita un proveedor que la implemente, como Hibernate.

JPA vs. JAKARTA

Mismo estándar, cambió el nombre del paquete

1. Hasta 2017 la especificación vivía en javax.persistence, como parte de Java EE (Oracle).

2. Oracle donó Java EE a la Eclipse Foundation, que no podía seguir usando la marca “Java”: nació Jakarta EE.

3. Desde Jakarta EE 9 (2020), los paquetes pasaron de javax.* a jakarta.*: por eso este curso usa jakarta.persistence.*.

```
import javax.persistence.*;     // Java EE (hasta 2019)
import jakarta.persistence.*;   // Jakarta EE 9+ (este curso)
```

“JPA” (Java Persistence API) quedó como apodo: el nombre oficial hoy es “Jakarta Persistence”.

IMPLEMENTACIONES DE JPA

HIBERNATE

El proveedor JPA más usado; en el que se basan los ejemplos de esta clase.

ECLIPSELINK

Implementación de referencia de la especificación, mantenida por la fundación Eclipse.

OPENJPA

Implementación de Apache, usada en algunos entornos empresariales.

SPRING DATA JPA

Una capa sobre JPA que genera automáticamente el DAO a partir de una interfaz.

AGREGAR HIBERNATE CON MAVEN

Tres dependencias en el pom.xml del proyecto

Las tres dependencias van dentro de <dependencies> en el pom.xml:

```
<dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.1.0</version>
</dependency>
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.6.22.Final</version>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>9.5.0</version>
</dependency>
```

ANT vs. MAVEN

ANT

- **Dependencias manuales**

Cada .jar (Hibernate, JPA, el driver...) se descarga y se agrega al proyecto a mano.

- **Sin resolución transitiva**

Si una librería necesita otras, hay que buscarlas e incluirlas una por una.

- **build.xml a medida**

Los pasos de compilación y empaquetado se escriben y se mantienen a mano.

MAVEN

- **Dependencias declaradas**

Cada librería se declara una vez en el pom.xml, con su versión exacta.

- **Resolución automática**

Maven también descarga las dependencias transitivas (lo que Hibernate necesita).

- **Estándar en proyectos Java**

Cualquiera con el mismo pom.xml compila el proyecto igual, sin instalar nada a mano.

FLUJO BÁSICO DE TRABAJO CON JPA

Los pasos que se repiten en cualquier operación

1. Configurar la unidad de persistencia en persistence.xml (conexión y proveedor).

2. Crear un EntityManagerFactory a partir de esa unidad de persistencia.

3. Obtener un EntityManager desde la factory.

4. Iniciar una transacción para operaciones que modifican datos.

5. Ejecutar la operación (persist, find, una consulta JPQL) y confirmar la transacción.

6. Cerrar el EntityManager (y el EntityManagerFactory al finalizar la aplicación).

CONFIGURACIÓN DE JPA



La unidad de persistencia describe cómo conectarse

```
<!-- src/main/resources/META-INF/persistence.xml -->
<persistence-unit name="escuelaPU">
    <properties>
        <property name="jakarta.persistence.jdbc.url"
            value="jdbc:mysql://localhost:3306/escuela"/>
        <property name="jakarta.persistence.jdbc.user" value="root"/>
        <property name="jakarta.persistence.jdbc.password" value="clave"/>
        <property name="hibernate.hbm2ddl.auto" value="update"/>
    </properties>
</persistence-unit>

// Una sola vez, al iniciar la aplicación (por ejemplo, en el main):
EntityManagerFactory emf =
    Persistence.createEntityManagerFactory("escuelaPU");
EntityManager em = emf.createEntityManager();
```

LA ENTIDAD PERSONA CON JPA



Anotaciones que mapean la clase a la tabla personas

```
import jakarta.persistence.*;

@Entity
@Table(name = "personas")
public class Persona {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String apellido;
    private String documento;

    // Constructores y getters en la próxima diapositiva
}
```

CONSTRUCTORES Y GETTERS

JPA exige un constructor vacío para instanciar la entidad

```
public Persona() {
    // JPA necesita este constructor sin argumentos
}

public Persona(String nombre, String apellido, String documento) {
    this.nombre = nombre;
    this.apellido = apellido;
    this.documento = documento;
}

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public String getDocumento() { return documento; }
```

ANOTACIONES BÁSICAS DE JPA

Cómo una clase se convierte en una tabla

`@Entity`
Marca la clase como una entidad persistente (una fila de una tabla).

`@Table(name = "...")`
Indica el nombre de la tabla si es distinto del nombre de la clase.

`@Id`
Marca el atributo que es la clave primaria de la entidad.

`@GeneratedValue`
Indica que el valor de la clave primaria lo genera la base de datos.

`@Column(name = "...")`
Mapea un atributo a una columna con otro nombre (o ajusta tipo, longitud, etc.).

Documentación oficial de las anotaciones (Jakarta Persistence 3.1): jakarta.ee/specifications/persistence/3.1/apidocs

ANOTACIONES DE RELACIONES

Para cuando una entidad se relaciona con otra

`@ManyToOne`
Varios registros de esta entidad se relacionan con uno de otra (ej: muchos Pedidos, un Cliente).

`@OneToMany`
El lado inverso de @ManyToOne: una entidad tiene una colección de otra (usa mappedBy).

`@JoinColumn`
Define el nombre de la columna que guarda la clave foránea en la tabla.

`@ManyToMany`
Relación de muchos a muchos; JPA la resuelve con una tabla intermedia.

La entidad Persona de este curso no usa relaciones: quedan para más adelante.

Documentación oficial de las anotaciones (Jakarta Persistence 3.1): jakarta.ee/specifications/persistence/3.1/apidocs

INSERTAR UNA PERSONA CON JPA



Alta de una Persona con EntityManager

```
EntityManager em = emf.createEntityManager();

Persona p = new Persona("Juan", "Pérez", "30111222");

em.getTransaction().begin();
em.persist(p);
em.getTransaction().commit();

System.out.println("Id generado: " + p.getId());

em.close();
```

LISTADO DE PERSONAS CON JPA



Consulta JPQL con EntityManager

```
EntityManager em = emf.createEntityManager();

List<Persona> personas =
    em.createQuery("SELECT p FROM Persona p", Persona.class)
      .getResultList();

for (Persona p : personas) {
    System.out.println(p.getApellido() + ", " + p.getNombre());
}

em.close();
```

CONSULTAS JPQL CON WHERE

Parecido a SQL, pero trabaja sobre entidades

JPQL (Java Persistence Query Language) es como SQL, pero orientado a objetos: en vez de tablas y columnas, usa entidades y atributos.

Los parámetros con nombre (:apellido) evitan concatenar valores a mano y previenen inyección SQL.

```
TypedQuery<Persona> query = em.createQuery(
    "SELECT p FROM Persona p WHERE p.apellido = :apellido",
    Persona.class);
query.setParameter("apellido", "Pérez");

List<Persona> personas = query.getResultList();
```

Así se implementaría findByLastName() en PersonaDAOJPA.

NAMED QUERIES

JPQL reutilizable, declarado junto a la entidad

Se declara una sola vez, arriba de la entidad:

```
@Entity
@NamedQuery(
    name = "Persona.buscarPorApellido",
    query = "SELECT p FROM Persona p WHERE p.apellido = :apellido")
public class Persona { /* ... */ }
```

Y se usa por nombre, sin repetir el JPQL:

```
TypedQuery<Persona> query =
    em.createNamedQuery("Persona.buscarPorApellido", Persona.class);
query.setParameter("apellido", "Pérez");
```

JDBC vs. JPA

JDBC

- **Control total del SQL**

Cada consulta se escribe a mano; útil para casos muy específicos o de alto rendimiento.

- **Mapeo manual**

Hay que armar cada objeto fila por fila desde el ResultSet.

- **Más código repetitivo**

Conexión, sentencia y mapeo se repiten en cada operación.

JPA

- **Mapeo automático**

Las anotaciones describen la correspondencia entre clase y tabla.

- **Menos código**

persist(), find() y JPQL reemplazan buena parte del código repetitivo de JDBC.

- **Portabilidad**

Cambiar de motor de base de datos implica, en general, mucho menos trabajo.

INTERFAZ GENÉRICA DAO<T, ID>



Repaso: el contrato no cambia

```
public interface DAO<T, ID> {

    void insert(T entity) throws DaoException;

    T findById(ID id) throws DaoException;

    List<T> findAll() throws DaoException;

    void update(T entity) throws DaoException;

    void delete(ID id) throws DaoException;
}
```

INTERFAZ PERSONADAO



Repaso: especialización de DAO<T, ID> para Persona

```
public interface PersonaDAO extends DAO<Persona, Long> {

    List<Persona> findByLastName(String lastName) throws DaoException;
}
```

IMPLEMENTACIÓN JPA: BUSCAR ID

PersonaDAOJPA — buscar por id

```
public class PersonaDAOJPA implements PersonaDAO {
    private final EntityManager em;

    public PersonaDAOJPA(EntityManager em) {
        this.em = em;
    }

    @Override
    public Persona findById(Long id) throws DaoException {
        try {
            return em.find(Persona.class, id);
        } catch (PersistenceException e) {
            throw new DaoException("No se pudo buscar la persona con id " + id, e);
        }
    }
}
```

IMPLEMENTACIÓN JPA: INSERTAR



PersonaDAOJPA — insertar

```
@Override
public void insert(Persona p) throws DaoException {
    EntityTransaction tx = em.getTransaction();
    try {
        tx.begin();
        em.persist(p);
        tx.commit();
    } catch (PersistenceException e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw new DaoException("No se pudo insertar la persona", e);
    }
}
```

IMPLEMENTACIÓN JPA: LISTAR



PersonaDAOJPA — listar todos

```
@Override
public List<Persona> findAll() throws DaoException {
    try {
        TypedQuery<Persona> query = em.createQuery(
            "SELECT p FROM Persona p", Persona.class);
        return query.getResultList();
    } catch (PersistenceException e) {
        throw new DaoException("No se pudo listar las personas", e);
    }
}
```

VENTAJAS DE JPA



MENOS CÓDIGO

persist(), find() y las consultas JPQL reemplazan buena parte del código repetitivo de JDBC.

MAPEO DECLARATIVO

Las anotaciones describen la relación entre clases y tablas, en vez de armarla a mano.

PORTABILIDAD

Cambiar de motor de base de datos requiere, en general, tocar mucho menos código.

CACHÉ Y OPTIMIZACIONES

El proveedor JPA puede cachear entidades y optimizar el acceso sin que el código lo note.



MENSAJE FINAL

"La mejor herramienta es la que te deja pensar en el dominio, no en el SQL."

JPA no reemplaza a JDBC: lo usa por debajo, pero te permite programar contra objetos en lugar de contra tablas.